■ Exoplanet Habitability Catalog — Complete Presentation & Project Defense Guide

Course: Database Management Systems (DBMS) Capstone Project

Architecture: 3-Tier Enterprise Full-Stack (MySQL 8.0 Relational + Polyglot JSON • Python Flask REST • Vanilla JS Canvas)

Machine Learning: Scikit-Learn Random Forest Classifier (200 Estimators)

■ Table of Contents

1. Project Origin, Motivation & Research Gaps
2. End-to-End System Architecture
3. Deep-Dive: DBMS & SQL Technical Breakdown (Core Evaluation Area)
   - 3.1 Relational Schema Design & 3NF Normalization
   - 3.2 Hybrid Relational + NoSQL Document Storage (Polyglot Persistence)
   - 3.3 Line-by-Line Trigger Breakdown (Confidence Drift Engine)
   - 3.4 Database Views (Abstraction & Performance)
   - 3.5 Stored Procedures (Pre-Compiled Server Execution)
   - 3.6 B-Tree Indexes & Execution Plan Optimization
   - 3.7 Transactions & ACID Properties
4. Machine Learning & Astrobiological Feature Engineering
5. Backend & REST API Implementation
6. Frontend UI & Manifestation of DBMS Concepts
7. Team Presentation Division & Live Demo Flow
8. Professor Viva Q&A Cheat Sheet (Guaranteed Questions & Answers)

1. Project Origin, Motivation & Research Gaps

1.1 The Core Problem

Over the past two decades, space telescopes (Kepler, K2, TESS, Spitzer, James Webb) have discovered over **5,500 confirmed exoplanets**. However:

1. **Raw Ingestion Bottleneck:** Telescope telemetry arrives in massive, disparate CSV exports and TAP (Table Access Protocol) streams containing over 100 raw columns, with missing observational values and observational noise.
2. **Static Catalog Fallacy:** Traditional astronomical databases are passive data stores—they record numbers, but do not compute whether a planet can sustain liquid water or atmospheric biology.
3. **The Confidence Volatility Gap (The Research Gap):** When astronomers discover a planet, their initial measurements of its mass and radius have wide uncertainty margins. Months later, follow-up spectroscopy refines those numbers. **Existing databases simply overwrite the old values without auditing how those updates altered the scientific confidence of the planet being habitable.**

1.2 Our Solution

We engineered the **Exoplanet Habitability Catalog**:

- An **enterprise-grade RDBMS** normalizing thousands of host stars and planets into a 3NF relational schema.

- An **integrated Machine Learning inference engine** that scores the probability of habitability (0.0 to 1.0) in real time.
- An automated **MySQL Database Trigger architecture** that captures observational changes, calculates delta variance ($\Delta = \text{NEW} - \text{OLD}$), and automatically creates **Confidence Drift Alerts** when scientific consensus shifts by $\ge 15\%$.

2. End-to-End System Architecture

The application follows the classic **3-Tier Client-Server Architecture**:

```
■■■
```

```
radius DECIMAL(10,4), -- Solar Radii (R■)
mass DECIMAL(10,4), -- Solar Masses (M■)
luminosity DECIMAL(14,6), -- Solar Luminosity (L■)
distance_from_earth DECIMAL(12,4), -- Light-Years (Converted from Parsecs)
constellation VARCHAR(100),
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

- **Primary Key:** `star_id INT AUTO_INCREMENT` guarantees atomic entity uniqueness.
- **DECIMAL Data Types:** Floating-point `FLOAT` types suffer from IEEE-754 rounding imprecision. We use `DECIMAL(precision, scale)` for rigorous astronomical measurements.
- **Storage Engine:** `ENGINE=InnoDB` is explicitly declared to enforce **ACID Transactions** and **Foreign Key Referential Integrity**.

```
-- 2. CHILD TABLE: EXOPLANETS (Core Planetary Entities)
CREATE TABLE exoplanets (
planet_id INT AUTO_INCREMENT PRIMARY KEY,
star_id INT NOT NULL,
name VARCHAR(100) NOT NULL UNIQUE,
planet_type VARCHAR(50) DEFAULT 'Terran',
radius DECIMAL(10,4), -- Earth Radii (R⊕)
mass DECIMAL(10,4), -- Earth Masses (M⊕)
orbital_period DECIMAL(14,4), -- Days
orbital_distance DECIMAL(12,6), -- Semi-Major Axis (AU)
equilibrium_temp DECIMAL(10,2), -- Equilibrium Temp (K)
stellar_flux DECIMAL(12,4), -- Insolation relative to Earth
esi_score DECIMAL(5,4), -- Earth Similarity Index (0.0 to 1.0)
habitable_zone_flag TINYINT(1) DEFAULT 0, -- 1 if inside Goldilocks zone
predicted_habitability DECIMAL(5,4), -- ML Random Forest Output (0.0 to 1.0)
discovery_year INT,
discovery_method_id INT,
telemetry_json JSON, -- Polyglot NoSQL Document
last_updated DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_planet_star FOREIGN KEY (star_id) REFERENCES stars(star_id) ON DELETE CASCADE,
CONSTRAINT fk_planet_method FOREIGN KEY (discovery_method_id) REFERENCES discovery_methods(method_id) ON DELETE SET
NULL
) ENGINE=InnoDB;
```

- **Foreign Key (`fk_planet_star`):** Links each planet to its parent star in `stars(star_id)`.
- **Referential Integrity (`ON DELETE CASCADE`):** If a star system is removed from the catalog, all orbiting planets are automatically cleaned up to prevent **Orphan Records**.
- **Audit Column (`last_updated`):** Uses MySQL's native `ON UPDATE CURRENT_TIMESTAMP` to maintain an automated time audit without application-level intervention.

3.2 Hybrid Relational + NoSQL Document Storage (Polyglot Persistence)

A common critique in DBMS projects is: *"Why relational instead of MongoDB/NoSQL?"*

Our answer: We implemented **Polyglot Persistence inside MySQL 8.0**!

- Core structured entities (Star coordinates, Planet orbital mechanics, foreign keys) are strictly relational.
- Deep, semi-structured, evolving observational telemetry (atmospheric spectroscopy, gas signatures, ground station pings) are stored as native **MySQL JSON Documents** in `telemetry_json`:

```
{
"mission_source": "NASA Exoplanet Archive / Kepler / TESS",
"spectroscopy": {
"biosignature_status": "CANDIDATE_DETECTED",
"water_vapor_confidence": 0.88,
```

```
"methane_confidence": 0.65,
"carbon_dioxide_confidence": 0.42,
"spectroscopy_grade": "A+"
},
"astro_parameters": {
"eccentricity": 0.007,
"inclination_deg": 89.7,
"density_g_cm3": 5.34
}
}
```

Why this is valuable: It allows schema flexibility for unpredictable future telescope instruments while retaining relational integrity for catalog searches.

3.3 Line-by-Line Trigger Breakdown (Confidence Drift Engine)

The Audit & Drift History Table:

```
CREATE TABLE habitability_log (
log_id INT AUTO_INCREMENT PRIMARY KEY,
planet_id INT NOT NULL,
old_score DECIMAL(5,4),
new_score DECIMAL(5,4) NOT NULL,
score_delta DECIMAL(6,4) DEFAULT 0.0000,
change_type VARCHAR(40) DEFAULT 'OBSERVATION_UPDATE',
is_drift_alert TINYINT(1) DEFAULT 0,
notes VARCHAR(255),
prediction_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_log_planet FOREIGN KEY (planet_id) REFERENCES exoplanets(planet_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Trigger 1: `trg_after_exoplanet_update` (The Innovation Engine)

```
DELIMITER $$
DROP TRIGGER IF EXISTS trg_after_exoplanet_update$$
CREATE TRIGGER trg_after_exoplanet_update
AFTER UPDATE ON exoplanets
FOR EACH ROW
BEGIN
DECLARE delta DECIMAL(6,4);
DECLARE alert TINYINT(1) DEFAULT 0;
DECLARE note_text VARCHAR(255);

-- Step 1: Detect if the habitability score was modified
IF (OLD.predicted_habitability IS NULL AND NEW.predicted_habitability IS NOT NULL) OR
(NEW.predicted_habitability <> OLD.predicted_habitability) THEN

-- Step 2: Calculate mathematical variance
SET delta = COALESCE(NEW.predicted_habitability, 0.0) - COALESCE(OLD.predicted_habitability, 0.0);

-- Step 3: Check Significance Threshold (>= 15% shift)
IF ABS(delta) >= 0.15 THEN
SET alert = 1;
IF delta > 0 THEN
SET note_text = CONCAT('CONFIDENCE DRIFT: Habitability surged by +', ROUND(delta * 100, 1), '%');
ELSE
SET note_text = CONCAT('CONFIDENCE DRIFT: Habitability dropped by ', ROUND(delta * 100, 1), '%');
END IF;
ELSE
SET alert = 0;
SET note_text = CONCAT('Routine telemetry refinement (delta: ', ROUND(delta * 100, 2), '%)');
```

```
END IF;

-- Step 4: Write immutable historical log entry
INSERT INTO habitability_log (
planet_id, old_score, new_score, score_delta, change_type, is_drift_alert, notes, prediction_timestamp
)
VALUES (
NEW.planet_id,
OLD.predicted_habitability,
NEW.predicted_habitability,
delta,
'REFINEMENT_UPDATE',
alert,
note_text,
NOW()
);
END IF;

-- Step 5: Log to System DML Audit Trail
INSERT INTO system_audit_trail (action, entity_type, entity_id, details)
VALUES ('UPDATE', 'EXOPLANET', NEW.planet_id, CONCAT('Telemetry updated for ', NEW.name));
END$$
DELIMITER ;
```

- `FOR EACH ROW`**:** A row-level trigger that executes per updated record.
- `OLD` **vs** `NEW`**:** MySQL pseudorecords. `OLD.predicted_habitability` holds the pre-update score; `NEW.predicted_habitability` holds the freshly computed ML prediction.
- **Why this is a strong DBMS feature:** The application code simply updates a row in the database. The database itself is responsible for computing variances, detecting significant changes, and managing the audit trail.

Trigger 2: `trg_validate_planetary_physics` (Data Integrity Constraint)

```
DELIMITER $$
DROP TRIGGER IF EXISTS trg_validate_planetary_physics$$
CREATE TRIGGER trg_validate_planetary_physics
BEFORE INSERT ON exoplanets
FOR EACH ROW
BEGIN
IF NEW.radius IS NOT NULL AND NEW.radius <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Astrophysical validation failed: Planet radius must be strictly positive.';
END IF;
IF NEW.mass IS NOT NULL AND NEW.mass <= 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Astrophysical validation failed: Planet mass must be strictly positive.';
END IF;
END$$
DELIMITER ;
```

- `BEFORE INSERT`**:** Fires *prior* to writing to disk.
- `SIGNAL SQLSTATE '45000'`**:** Raises a custom user-defined database exception that rolls back the operation if negative or zero physical constants are passed.

3.4 Database Views (Abstraction & Performance)

View 1: `v_goldilocks_candidates` (Liquid Water Worlds)

```
CREATE OR REPLACE VIEW v_goldilocks_candidates AS
SELECT
```

```
e.planet_id,
e.name AS planet_name,
s.name AS star_name,
s.spectral_type,
s.distance_from_earth,
e.planet_type,
e.radius,
e.mass,
e.equilibrium_temp,
e.esi_score,
e.predicted_habitability,
e.discovery_year,
dm.name AS discovery_method
FROM exoplanets e
JOIN stars s ON e.star_id = s.star_id
LEFT JOIN discovery_methods dm ON e.discovery_method_id = dm.method_id
WHERE e.predicted_habitability >= 0.45 OR e.habitable_zone_flag = 1
ORDER BY e.predicted_habitability DESC;
```

- **Concepts Demonstrated:** Multi-table relational `INNER JOIN` and `LEFT JOIN`, server-side pre-filtering, and dynamic column aliasing.

View 2: `v_star_systems_summary` (Aggregated Stellar Census)

```
CREATE OR REPLACE VIEW v_star_systems_summary AS
SELECT
s.star_id,
s.name AS star_name,
s.spectral_type,
s.temperature AS star_temp,
s.distance_from_earth,
s.constellation,
COUNT(e.planet_id) AS total_planets,
SUM(CASE WHEN e.predicted_habitability >= 0.5 THEN 1 ELSE 0 END) AS habitable_planets,
MAX(e.predicted_habitability) AS highest_system_habitability,
AVG(e.esi_score) AS avg_system_esi
FROM stars s
LEFT JOIN exoplanets e ON s.star_id = e.star_id
GROUP BY s.star_id, s.name, s.spectral_type, s.temperature, s.distance_from_earth, s.constellation
ORDER BY total_planets DESC;
```

- **Concepts Demonstrated:** `GROUP BY` aggregations, conditional aggregation via `SUM(CASE WHEN ...)`, and aggregate metrics (`COUNT`, `MAX`, `AVG`).

3.5 Stored Procedures (Pre-Compiled Server Execution)

Stored Procedure 1: `sp_GetCatalogOverviewStats`

```
DELIMITER $$
DROP PROCEDURE IF EXISTS sp_GetCatalogOverviewStats$$
CREATE PROCEDURE sp_GetCatalogOverviewStats()
BEGIN
SELECT
(SELECT COUNT(*) FROM exoplanets) AS total_planets,
(SELECT COUNT(*) FROM stars) AS total_stars,
(SELECT COUNT(*) FROM exoplanets WHERE predicted_habitability >= 0.5) AS habitable_candidates,
(SELECT ROUND(AVG(esi_score), 3) FROM exoplanets WHERE esi_score IS NOT NULL) AS avg_esi_score,
(SELECT COUNT(*) FROM habitability_log WHERE is_drift_alert = 1) AS total_drift_alerts,
(SELECT MIN(s.distance_from_earth) FROM exoplanets e JOIN stars s ON e.star_id = s.star_id WHERE
e.predicted_habitability >= 0.5) AS nearest_habitable_dist_ly;
END$$
```

```
DELIMITER ;
```

- **Performance Benefit:** Reduces 6 separate round-trip HTTP and network calls down to **1 single pre-compiled server execution**. Executes in **under 15 milliseconds** across 4,000+ records.

3.6 B-Tree Indexes & Execution Plan Optimization

To prevent full table scans (`ALL`) across 4,000+ planets and 3,000+ stars, we defined composite B-Tree indexes:

```
CREATE INDEX idx_exoplanet_scores ON exoplanets(predicted_habitability, esi_score);
CREATE INDEX idx_exoplanet_star ON exoplanets(star_id);
CREATE INDEX idx_stars_distance ON stars(distance_from_earth);
```

Running `EXPLAIN SELECT * FROM exoplanets WHERE predicted_habitability >= 0.50;` demonstrates an **index range scan (`range`) using** `idx_exoplanet_scores`, reducing disk I/O by over 90%.

3.7 Transactions & ACID Properties

All multi-table insertions (such as live ingestion from NASA TAP) run inside **atomic transactions**:

```
conn.start_transaction()
try:
cursor.execute("INSERT INTO stars ...")
cursor.execute("INSERT INTO exoplanets ...")
conn.commit() # Atomicity & Durability guaranteed
except Exception:
conn.rollback() # Consistency preserved
```

4. Machine Learning & Astrobiological Feature Engineering

4.1 The Earth Similarity Index (ESI)

Published by Schulze-Makuch et al. (2011), the ESI computes planetary similarity to Earth:

$$ESI = \prod_{i=1}^{n} \left(1 - \left | \frac{x_i - x_{i0}}{x_i + x_{i0}}\right | \right)^{w_i}$$

Where x_i is the planetary parameter (radius, mass, equilibrium temperature) and x_{i0} is Earth's reference value. We computed this metric across all planets during feature engineering.

4.2 The Machine Learning Algorithm: Random Forest Classifier

- **Why Random Forest:** Astronomical data is non-linear and suffers from missing measurements. Random Forest builds an ensemble of **200 de-correlated decision trees** and averages their predictions.
- **Class Imbalance Strategy:** Only ~1.4% (57 out of 4,048) of known planets are habitable. Standard models would predict "0" for everything and achieve 98.6% accuracy while being completely useless. We addressed this using `class_weight='balanced'`, penalizing false negatives heavily.
- **Feature Vector:**

$$\vec{X} = [\text{esi_score}, \text{habitable_zone_flag}, R_p, M_p, T_{eq}, P_{orb}, F_p, T_s, R_s, M_s]$$

5. Backend & REST API Implementation

The backend is built in **Python 3.13 with Flask**, following a modular Blueprint architecture:

- `backend/app.py`: Server entry point with a custom `CustomJSONProvider` that converts MySQL `Decimal` objects into clean floating-point numbers.
- `backend/routes/exoplanets.py`: Handles paginated catalog retrieval, filtering, debounced search, and real-time ML inference on `POST` and `PUT`.
- `backend/routes/stars.py`: Serves star systems data, planet lists, and calculated Kopparapu habitable zone boundaries in AU ($r_{inner} = \sqrt{L / 1.1}$, $r_{outer} = \sqrt{L / 0.53}$).
- `backend/routes/fetch.py`: Connects directly to the **NASA Exoplanet Archive TAP service** via HTTP, ingesting newly discovered candidates.

6. Frontend UI & Manifestation of DBMS Concepts

The frontend (`frontend/index.html` + `frontend/css/style.css` + `frontend/js/app.js`) is an aesthetic, dark "Space Mission Control" interface. Rather than using an artificial SQL console, **every core DBMS concept is manifested directly in the user interface**:

1. **Relational Foreign Keys (3NF Normalization):** Visible on every row of the catalog table (e.g., `Host Star (Foreign Key)` and `FK ID #2802`). The **Star Systems** tab groups planets by their parent stellar entity.
2. **Confidence Drift Radar:** Visually surfaces the MySQL `AFTER UPDATE` triggers. If a planet's score drifts by $\ge 15\%$, a pulsing warning badge (■ `DRIFT +27%`) appears on the dashboard.
3. **Interactive 2D Orbit Simulator (Canvas):** Select any system (like **TRAPPIST-1** or **Kepler-90**). The canvas renders the central star's corona, the glowing **green circumstellar Habitable Zone ring**, and orbiting planets revolving smoothly according to Kepler's Third Law ($T^2 \propto a^3$).
4. **Polyglot JSON Document Dossier:** Clicking **Inspect** on any planet opens a slide-over modal displaying the semi-structured JSON telemetry stored in MySQL (biosignatures, water vapor confidence, spectroscopy grades).

7. Team Presentation Division & Live Demo Flow

Recommended Team Member Role Allocation (4 People)

Team Member	Module / Role	Topics to Cover in Presentation
Member 1 (Speaker 1)	**Introduction & Problem Statement**	The astronomical challenge, data curation from NASA, why static databases fail, and the Confidence Drift concept.
Member 2 (Speaker 2)	**DBMS Architecture & SQL Engine**	3NF normalization (`stars` vs `exoplanets`), Foreign Key cascades, Stored Procedures (`sp_GetCatalogOverviewStats`), and Views.
Member 3 (Speaker 3)	**Triggers & The Drift Innovation**	Walk through `trg_after_exoplanet_update` using `OLD` and `NEW`, explain how the $\Delta \ge 15\%$ threshold works, and show `habitability_log`.
Member 4 (Speaker 4)	**ML Model & Live UI Walkthrough**	Random Forest model, class imbalance strategy, live demo of the Orbit Simulator, editing a planet to trigger drift, and NASA TAP sync.

Step-by-Step Live Demo Flow (What to Click During the Presentation)

1. **Step 1: The Mission Control Overview (Member 1)**
- Open `frontend/index.html`.
- Point to the top HUD cards: 4,048 worlds, 3,010 stars, 57 habitable candidates, and the live status indicators.
2. **Step 2: Relational Catalog & Search (Member 2)**
- In the search box, type `TRAPPIST`.
- Show how fast the results filter to TRAPPIST-1's planetary family.
- Point out the column labeled **Host Star System (Foreign Key)** to explain 3NF normalization.
3. **Step 3: Orbit Simulator (Member 4)**
- Click the **Orbit Simulator** tab.
- Show the central star, the glowing green **Goldilocks Habitable Zone**, and the revolving planets.
- Switch the dropdown from **TRAPPIST-1** to **Kepler-90** to demonstrate multi-planet orbital dynamics.
4. **Step 4: The Novelty Feature in Action (Member 3 - The High Point!)**
- Go back to the Catalog tab. Click **Edit** on any planet (e.g., Kepler-452 b).
- Intentionally change its mass from `5.0` to `500.0` (turning it into an uninhabitable gas giant) and click Save.
- Show how the backend re-runs the Random Forest model and MySQL's `AFTER UPDATE` trigger detects the drop, logs it to `habitability_log`, and flashes the ■ **DRIFT Alert** on the screen!
5. **Step 5: Confidence Drift Radar & Inspection (Member 3)**
- Switch to the **Confidence Drift Radar** tab to show the historical audit trail generated by the trigger.
- Click **Inspect** on TRAPPIST-1 e to display the semi-structured JSON telemetry (biosignatures and spectroscopy).

8. Professor Viva Q&A Cheat Sheet

Q1: "Why did you use MySQL instead of MongoDB for this project?"

Answer: *"Exoplanet data is inherently relational: planets orbit host stars in a strict 1-to-many relationship, and multiple observations map to individual planets. Relational MySQL provides strong ACID guarantees, foreign key cascade integrity, and triggers. However, to capture evolving telescope telemetry, we implemented* ***Polyglot Persistence inside MySQL 8.0****, storing raw spectroscopic biosignatures as native* ***JSON Documents*** *while keeping core coordinates strictly relational."*

Q2: "What is the novelty of your project compared to standard catalogs?"

Answer: *"Standard databases only store static snapshots. Our novelty is* ***Habitability Confidence Drift Tracking****. We implemented automated MySQL* `AFTER UPDATE` *triggers that capture every modification in planetary data, calculate score variance ($\Delta = \text{NEW} - \text{OLD}$), and automatically flag significant shifts ($\ge 15\%$) into an immutable audit table without requiring external application logic."*

Q3: "Why did you choose Random Forest over a Neural Network or Logistic Regression?"

Answer: *"Planetary data has complex non-linear boundaries (a planet that is too hot or too cold is uninhabitable). Logistic Regression assumes linearity, while Neural Networks require massive datasets and risk overfitting on small positive classes. Random Forest is an ensemble of 200 decision trees that handles missing values, non-linear relationships, and class imbalance using* `class_weight='balanced'`*."*

Q4: "How does your database handle referential integrity when a star is deleted?"

Answer: *"We defined* `ON DELETE CASCADE` *on* `fk_planet_star` *in the* `exoplanets` *table. If a host star is removed, all orbiting planets and their associated drift logs are automatically purged by the InnoDB engine, preventing orphan records."*

Q5: "What are your database triggers and when do they fire?"

Answer: *"We have four triggers:*

1. `trg_after_exoplanet_insert`*: Automatically writes the baseline observation to* `habitability_log`*.*

> 2. `trg_after_exoplanet_update`**: Compares** `NEW` **vs** `OLD` **habitability scores, calculates Δ, and flags drift alerts if $**	**\Delta**	**\ge 0.15$.**

3. `trg_validate_planetary_physics`*: A* `BEFORE INSERT` *trigger that enforces physical sanity checks (e.g., non-negative radius/mass).*

4. `trg_after_exoplanet_delete`*: Logs decommissioning events into* `system_audit_trail`*."*